

Herald

Herald

Tagline: Every alert reaches the right destination — no command syntax required.

Summary: Herald ingests system events and reliably fans them out to multiple notification channels so every alert reaches the right place. Subscribers interact in plain natural language; Herald infers intent via a three-tier discipline (command fast-path → LLM intent inference → clarify fallback).

Short description (~40 words): An event ingestion and multi-channel notification fan-out system. Herald reliably routes system events to the right destinations across messenger channels, and lets subscribers speak plain natural language — resolving intent through a command fast-path, LLM inference, and a clarify-and-ask fallback.

Long description (150-250 words): Herald is the notification backbone that makes sure system events actually reach the humans and channels that need them. It ingests events and fans them out reliably to multiple notification channels, avoiding the common failure where alerts are dropped, mis-routed, or buried. What distinguishes Herald is its interaction model: subscribers do not learn a command syntax. They speak plain natural language, and Herald determines intent through a deliberate three-tier discipline — a fast-path that recognizes explicit commands, then LLM-based intent inference (via Claude Code) for free-form messages, and finally a clarify fallback that replies, tags, and asks when intent is ambiguous. This “recognize → infer → clarify” ladder keeps the common case instant while never guessing blindly. Herald also models participation and attribution: an operator-username env var (HERALD_<CHANNEL>_OPERATOR_USERNAME) and a participant/attribution contract drive created_by/assigned_to fields and notification @-tagging, so it is clear who did what and who is being notified. Governance-wise, Herald inherits the Helix Constitution as a co-located submodule and follows its rules, and it is an early production consumer of Docs Chain — its full 66-document Markdown→HTML/PDF/DOCX corpus is wired through Docs Chain exec: transforms and verifies clean. Herald is

primarily Shell/Go tooling with layered specifications (V1→V2→V3→V4 supersession) and per-channel operator setup guides for messengers and LLM/agent dispatchers.

Why we built it: Alerts fail quietly — sent to the wrong channel, dropped, or requiring rigid command syntax users won't remember. Herald was built to guarantee reliable fan-out and to let people respond in natural language, so notifications are both dependable and effortless to act on.

Why it's a game-changer: It merges reliable multi-channel event routing with natural-language intent resolution, so operators talk normally and the system figures out what they mean — with a safe clarify fallback instead of wrong guesses.

What's innovative: - Three-tier intent discipline: command fast-path → LLM inference → clarify-and-ask. - Natural-language subscriber interaction (no command syntax to learn). - Participant-attribution contract driving `created_by/assigned_to` + `@`-tagging. - Real Docs Chain consumer (66-doc corpus, multi-format, verified).

Biggest technical challenges + how solved: - *Ambiguous natural-language intent*: solved with the three-tier recognize/infer/clarify ladder rather than blind guessing. - *Reliable fan-out*: solved with an ingestion→multi-channel dispatch design so alerts reach the correct destination. - *Correct attribution across channels*: solved with the operator-username env var and participant-attribution contract. - *Documentation drift*: solved by wiring its docs corpus through Docs Chain with verified transforms.

Tech stack (why + how): - **Go** — core event/dispatch logic (per org language patterns). - **Shell** — operator tooling and setup scripts. - **Claude Code (LLM)** — intent inference tier for free-form messages. - **Messenger channel adapters** — multi-channel notification fan-out. - **Docs Chain** — documentation build/verify pipeline (Markdown→HTML/PDF/DOCX). - **Helix Constitution submodule** — inherited governance/rules.

Public links: - GitHub (vasic-digital): <https://github.com/vasic-digital/Herald> (public). - Authoritative contracts referenced in-repo: docs/design/INTENT_RECOGNITION.md, docs/design/PARTICIPANT_ATTRIBUTION.md, docs/guides/MESSENGER_CHANNELS.md (private to repo; public on GitHub).

Suggested diagrams/illustrations (OpenDesign): 1. Fan-out topology: event source → Herald → many channels. 2. Three-tier intent ladder (command → LLM inference → clarify). 3. Attribution flow: message → participant contract → `created_by/assigned_to` + `@`-tag. 4. Docs Chain integration for Herald's doc corpus.

Site relevance: vasic.digital (AI-assisted operations / notifications). Fits an "AI in the loop" infrastructure story.

Priority tier: vasic-util-secondary

Source provenance: gh repo view vasic-digital/Herald README (mission, three-tier intent, participant attribution, Constitution inheritance, Docs Chain consumer note, revision history); analysis/github-vasic-digital.md (description, size); Docs Chain README (Herald as first downstream consumer).